

Indicaciones específicas:

- Esta evaluación contiene 8 páginas (incluyendo esta página) con 3 preguntas. El total de puntos es 20.
- El tiempo límite para la evaluación es 100 minutos.
- Cada pregunta deberá ser respondida en un archivo diferente. Se deberá usar los siguientes nombres de archivos:
 - p1.cpp
 - p2.cpp
 - p3.cpp
- Deberá subir sus archivos directamente a www.gradescope.com.
- Recuerde que Gradescope solo conserva el último envío que se realiza. Si va a modificar su entrega, debe volver a adjuntar todos los archivos de su práctica.

Competencias:

- Para los alumnos de la carrera de Ciencia de la Computación
 - Aplicar conocimientos de computación apropiados para la solución de problemas definidos y sus requerimientos en la disciplina del programa. (Evaluar)
 - Analizar problemas e identificar los requerimientos computacionales apropiados para su solución. (Usar)
 - Usar técnicas y herramientas actuales necesarias para la práctica de la computación. (Usar)
- Para los alumnos de las carreras de Ingeniería
 - Capacidad de aplicar conocimientos de matemáticas (nivel 3)
 - Capacidad de aplicar conocimientos de ingeniería (nivel 2)
 - Capacidad para diseñar un sistema, un componente o un proceso para satisfacer necesidades deseadas dentro de restricciones realistas (nivel 2)

Calificación:

Tabla de puntos (Sólo para uso del profesor)

Question	Points	Score
1	7	
2	6	
3	7	
Total:	20	

Preguntas

1. (7 puntos) Matrices dinámicas:

Usted deberá crear una matriz dinámica (puntero a puntero) de alto y ancho variable (valores ingresados por teclado).

En seguida, deberá rellenar todos los elementos de su matriz dinámica con valores '-' o '#' según la siguiente probabilidad: **existe un 0.67 de probabilidad de que cada celda sea una '-' y un 0.33 de probabilidad de que la celda sea un '#'.**

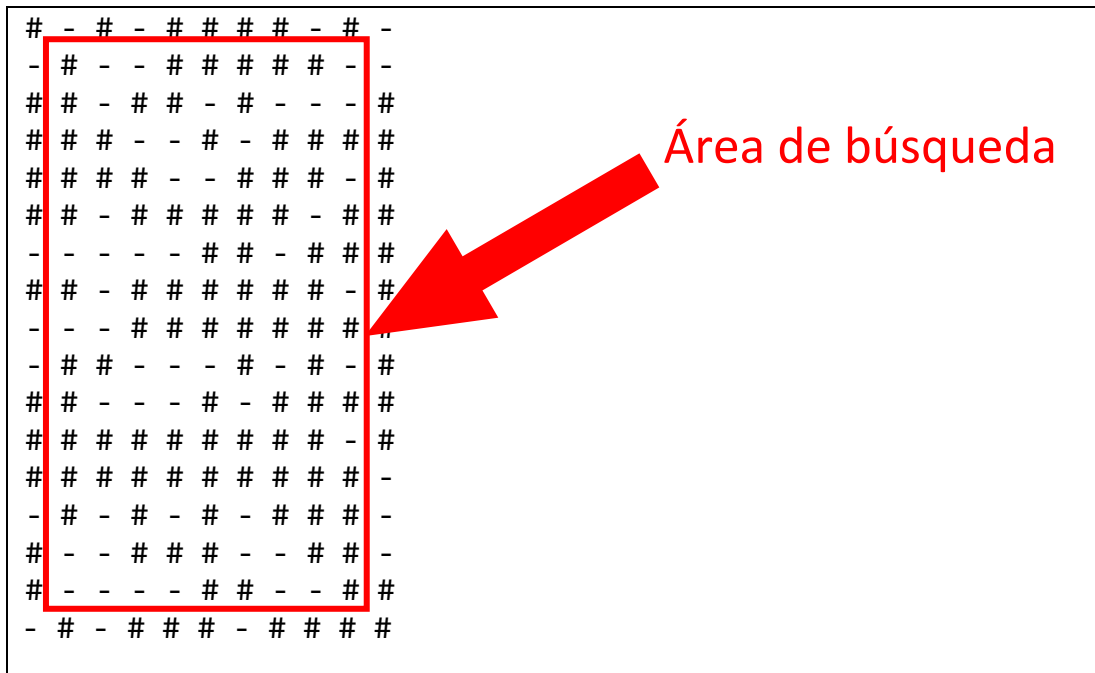
Una vez que haya llenado toda su matriz, debe imprimir toda la matriz en pantalla. Para que su matriz se vea más comprensible, por favor, **agregue un espacio entre cada carácter** que imprima.

Por ejemplo:

```
# - # - # # # - # -  
- # - - # # # # - -  
# # - # # - # - - - #  
# # # - - # - # # # #  
# # # # - - # # # - #  
# # - # # # # # - # #  
- - - - - # # - # # #  
# # - # # # # # - #  
- - - # # # # # # #  
- # # - - - # - # - #  
# # - - - # - # # # #  
# # # # # # # # - #  
# # # # # # # # # -  
- # - # - # - # # # -  
# - - # # # - - # # -  
# - - - - # # - - # #  
- # - # # # - # # # #
```

Finalmente, usted deberá buscar **todas** las casilla que tenga el carácter '-' que esté completamente selladas por las grillas (es decir, que los 8 vecinos de todo alrededor del guión sean grillas) y reemplazar dicha casilla por una letra O mayúscula.

Dado que es imposible que una casilla del borde esté rodeada por 8 grillas, solo busque en el interior de la matriz, excluyendo la primera y última columna, y excluyendo la primera y última fila, según se muestra a continuación:



Imprima la matriz resultante y el total de casillas completamente selladas. En nuestro ejemplo, la segunda impresión luciría de esta forma:



En caso ninguno de los guiones esté rodeado por 8 grillas, no habrá ningún reemplazo y, entonces, solo imprimiría dos veces la misma matriz.

Los criterios en la rúbrica (y el puntaje respectivo) se condicionan a que la solución presentada corresponda al problema planteado. El 100\% corresponde al puntaje indicado en cada punto.

Criterio	Excelente	Adecuado	Mínimo	Insuficiente
Ejecución	El diseño del algoritmo es ordenado y claro, siguiendo buenas prácticas en programación. La ejecución es correcta (100%).	El diseño del algoritmo es ordenado y claro. La ejecución es correcta (70%).	El diseño tiene algunas deficiencias pero la ejecución es correcta (30%).	El diseño es deficiente y la ejecución no es correcta (0%).
Sintaxis	No existen errores sintácticos o de compilación (100%).	Existen algunos errores sintácticos de menor relevancia, que no afectan el resultado (50%).	Existen errores sintácticos en la forma de ejecución, que no afectan el resultado (30%).	El código tiene errores de sintaxis que afectan el resultado (10%).
Optimización	El código es óptimo y eficiente. De buen performance e interacción con el usuario (100%).	El código es de buen performance durante la ejecución (70%).	El código no está optimizado pero la ejecución no es deficiente (30%).	El código no está optimizado y la ejecución es deficiente (0%).

2. (6 puntos) Vectores:

Usted deberá crear un vector de vectores de vectores de chars.

Dicho vector de vectores de vectores de chars se controlará con el índice i y tendrá tamaño 100 (como si fuera un edificio de 100 pisos).

Dentro de cada posición [i] de su edificio, usted creará un vector de vectores de chars controlado por el índice j, que tendrá tamaño 6, y dentro de cada posición [i][j] creará un vector de chars de tamaño 6, con todos los elementos inicializados en '-'.

Es decir, en cada piso [i] usted tendrá una matriz de 6x6 controlada por los índices [j][k].

Finalmente, usted deberá recorrer todo el edificio desde i=0 hasta i=99 haciendo que todas las matrices de 6x6 tengan el carácter '#' en su borde, tal y como se muestra en la Figura 1:

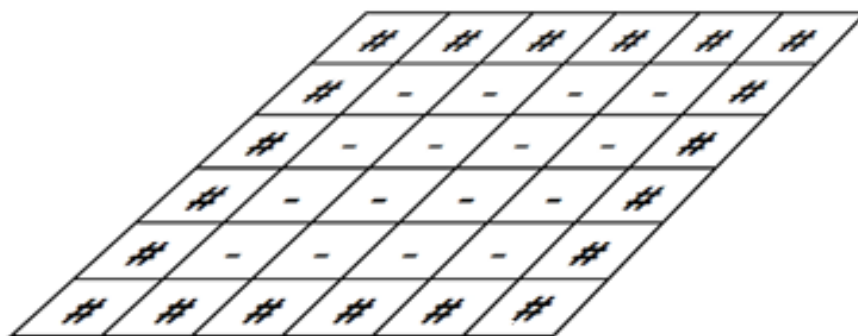


Figura 1: Una matriz de 6x6 con bordes

Imprima cada una de las 100 matrices tanto en la pantalla como en un archivo .txt, indicando también qué número de piso es. El primer piso, que se encuentra en el índice [0] deberá figurar como «Piso 1» y el último piso, que se encuentra en el índice [99] deberá figurar como «Piso 100».

Como resultado, su edificio tendrá paredes y estará hueco, tal y como se muestra en la Figura 2:

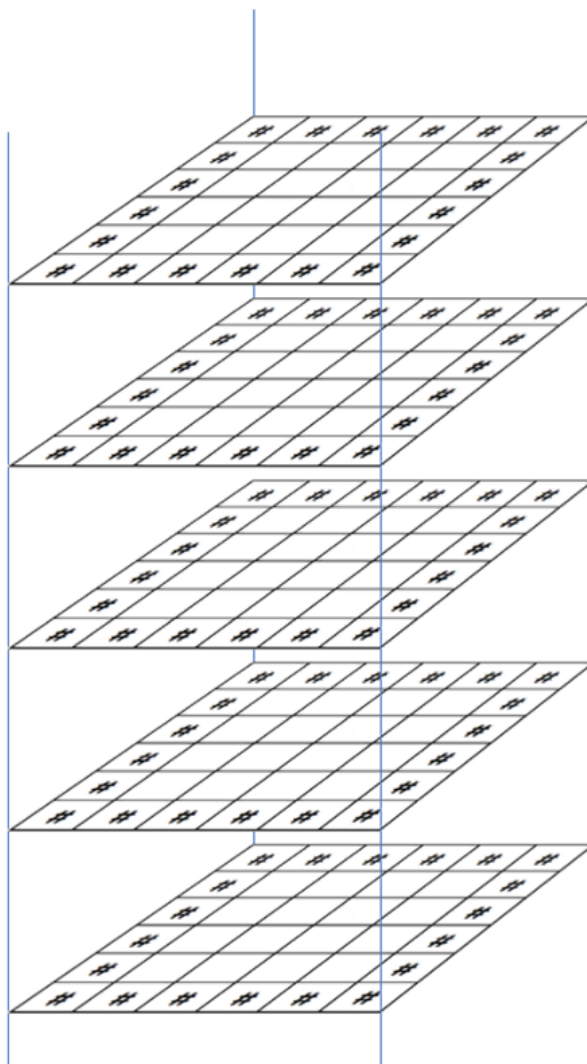


Figura 2: Una sección de su edificio (5 pisos de los 100 pisos)

A continuación, se muestra Un fragmento de la impresión que debe realizar su programa tanto en la pantalla como en un archivo .txt:

```
Piso 1:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# # # # # #

Piso 2:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# # # # # #

Piso 3:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# # # # # #

...

Piso 98:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# # # # # #

Piso 99:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# _ _ _ _ #
# # # # # #

Piso 100:
# # # # # #
# _ _ _ _ #
# _ _ _ _ #
```

```
# _ _ _ _ #
# _ _ _ _ #
# # # # # #
```

Usted debe imprimir los 100 pisos.

Los criterios en la rúbrica (y el puntaje respectivo) se condicionan a que la solución presentada corresponda al problema planteado. El 100\% corresponde al puntaje indicado en cada punto.

Criterio	Excelente	Adecuado	Mínimo	Insuficiente
Ejecución	El diseño del algoritmo es ordenado y claro, siguiendo buenas prácticas en programación. La ejecución es correcta (100%).	El diseño del algoritmo es ordenado y claro. La ejecución es correcta (70%).	El diseño tiene algunas deficiencias pero la ejecución es correcta (30%).	El diseño es deficiente y la ejecución no es correcta (0%).
Sintaxis	No existen errores sintácticos o de compilación (100%).	Existen algunos errores sintácticos de menor relevancia, que no afectan el resultado (50%).	Existen errores sintácticos en la forma de ejecución, que no afectan el resultado (30%).	El código tiene errores de sintaxis que afectan el resultado (10%).
Optimización	El código es óptimo y eficiente. De buen performance e interacción con el usuario (100%).	El código es de buen performance durante la ejecución (70%).	El código no está optimizado pero la ejecución no es deficiente (30%).	El código no está optimizado y la ejecución es deficiente (0%).

3. (7 puntos) Objetos:

Abra este video de youtube: <https://www.youtube.com/watch?v=yI2oS2hoL0k>

Implemente las clases y objetos necesarias para que ocurran todos los eventos que ve en el video. Debe imprimir todas las acciones tanto en la salida de la pantalla como en un archivo llamado eventos_del_video.txt.

Cada vez que un personaje haga una acción, debe especificar el minuto del video y el nombre del personaje (pueden llamarse ave1, ave2, personaje1, personaje2, detective, o como usted desee).

Por ejemplo:

- [Minuto 00:00] Una clase para el ave del comienzo que tenga el método **cantar()**, y un objeto ave en main que cante llamando al método **cantar()**. El resultado será un cout que dice «[minuto 00:00] Ave1 ha cantado».
- [Minuto 00:06] Una clase persona y 2 objetos de dicha clase que paseen en el coche (si desea, puede crear el objeto coche también, y ponerle un vector de pasajeros con dos objetos de la clase persona).
- [Minuto 00:10] 12 objetos de la clase persona. El primer objeto ejecuta el método **levantar_brazos()**, emitiendo un mensaje que dice «[minuto 00:12] personaje1 ha levantado los brazos», y luego ejecuta el método **asentirConLaCabeza()**, emitiendo un mensaje que dice «[minuto 0:13] personaje 1 asiente con la cabeza». Luego, los otros 11 personajes ejecutan el método **asentirConLaCabeza()**, emitiendo el mensaje 11 veces más (esta parte la puede hacer con una iterativa, iterando un vector de personajes o escribiendo la instrucción 11 veces, si es que no está usando vectores. Las 11 veces debe incluir el texto de [minuto 00:14].

Los criterios en la rúbrica (y el puntaje respectivo) se condicionan a que la solución presentada corresponda al problema planteado. El 100\% corresponde al puntaje indicado en cada punto.

Criterio	Excelente	Adecuado	Mínimo	Insuficiente
Ejecución	El diseño del algoritmo es ordenado y claro, siguiendo buenas prácticas en programación. La ejecución es correcta (100%).	El diseño del algoritmo es ordenado y claro. La ejecución es correcta (70%).	El diseño tiene algunas deficiencias pero la ejecución es correcta (30%).	El diseño es deficiente y la ejecución no es correcta (0%).
Sintaxis	No existen errores sintácticos o de compilación (100%).	Existen algunos errores sintácticos de menor relevancia, que no afectan el resultado (50%).	Existen errores sintácticos en la forma de ejecución, que no afectan el resultado (30%).	El código tiene errores de sintaxis que afectan el resultado (10%).
Optimización	El código es óptimo y eficiente. De buen performance e interacción con el usuario (100%).	El código es de buen performance durante la ejecución (70%).	El código no está optimizado pero la ejecución no es deficiente (30%).	El código no está optimizado y la ejecución es deficiente (0%).